Code Tweaks for the Relativity Toolkit

Brian Beckman

Jan 2026

Not for Publication to the Wolfram Community

In[•]:=
```
(*Quit[]*)
```

Relativity Toolkit

In[•]:=
```
(* ============================================================================*)
(* RELATIVITY TOOLKIT ENGINE (Script Mode) *)
(* Version: 1.6.1 (Feature: Riemann, fix display bug (A.Γ),μ) *)
(* ============================================================================*)
RelativityToolkitVersion = "1.6.1";

(* ============================================================================*)
(* 1. CLEAN SLATE ------------------------------------------------------------*)
(* ============================================================================*)

Unprotect[MakeBoxes];
Quiet[DownValues[MakeBoxes] =
   Select[DownValues[MakeBoxes], FreeQ[#, Partials] &]];
Quiet[DownValues[MakeBoxes] =
   Select[DownValues[MakeBoxes], FreeQ[#, CD] &]];
Quiet[MakeBoxes[δ[__], StandardForm] =.];
Quiet[MakeBoxes[Γ[__], StandardForm] =.];
Quiet[DownValues[MakeBoxes] =
   Select[DownValues[MakeBoxes],
    FreeQ[#, 𝒰] && FreeQ[#, 𝒟] &]];
Protect[MakeBoxes];

ClearAll[contractValence, valence, Partials, CanonicalizeTerm,
CanonicalizeIndices, TensorForm,
  CD, Γ, δ, upQ, downQ,
  λ, κ, ρ, σ, μ, ν, τ, η, χ, ψ,
  x, p, g, u, A, B, T,
  P, Q, S, metricRules, robustTransformRules, differentiationRules,
  ExpandDerivatives];

(* ============================================================================*)
(* 2. VALENCE LOGIC (The Type Checker) *)
```

```
(* ========================================================================*)

upQ[x_] := upQ[valence[x]];
downQ[x_] := downQ[valence[x]];
upQ[{{__}, {}}] := True; upQ[___] := False;
downQ[{{}, {__}}] := True; downQ[___] := False;

contractValence[{u_List, d_List}] :=
  Module[{common}, common = Intersection[u, d];
   {DeleteElements[u, common], DeleteElements[d, common]}];

valence[x_Symbol] := {{}, {}};
valence[_?NumericQ] := {{}, {}};
valence[] := Null;

valence[_[𝒰[α_]]] := {{α}, {}};
valence[_[𝒟[α_]]] := {{}, {α}};

valence[g[𝒟[μ_], 𝒟[ν_]]] := {{}, {μ, ν}};
valence[g[𝒰[μ_], 𝒰[ν_]]] := {{μ, ν}, {}};
valence[δ[𝒰[μ_], 𝒟[ν_]]] := {{μ}, {ν}};

valence[Partials[num_, den_]] :=
  Module[{vn, vd, vdFlipped}, vn = valence[num];
   vd = valence[den];
   vdFlipped = {vd⟦2⟧, vd⟦1⟧};
   {Join[vn⟦1⟧, vdFlipped⟦1⟧], Join[vn⟦2⟧, vdFlipped⟦2⟧]}];

valence[CD[num_, den_]] := valence[Partials[num, den]];

valence[Γ[𝒰[a_], 𝒟[
     b_], 𝒟[c_]]] := {{a}, {b, c}};

valence[Derivative[1][f_][arg_]] :=
  Module[{u, d}, {u, d} = valence[arg]; {d, u}];
valence[Derivative[_][_][f_] /; (valence[f] =!= {{}, {}})] :=
  valence[f];

valence[(h : _[𝒰[_]] | _[𝒟[_]])[___]] := valence[h];

valence[prod_Times] :=
  contractValence[
   Fold[Join[#1, valence[#2], 2] &, {{}, {}}, List @@ prod]];

valence[sum_Plus] :=
  Module[{vs}, vs = valence /@ (List @@ sum);
   If[Apply[SameQ, vs],
```

```
   First[vs],
   Print["Valence Mismatch"]; {{}, {}}]];

valence[Power[b_, _]] := valence[b];

valence[h_[a_, b_, rest___]] :=
  Join[valence[h[a]], valence[h[b, rest]], 2];

valence[___] := {{}, {}};

(* ========================================================================*)
(* 3. DISPLAY RULES *)
(* ========================================================================*)

Unprotect[MakeBoxes];

(*Semicolon Notation for CD*)
MakeBoxes[CD[num_, den_], StandardForm] :=
  If[MatchQ[den, x[𝒰[_]]],
   Replace[den,
    x[𝒰[idx_]] :>
     SubscriptBox[MakeBoxes[num, StandardForm],
      RowBox[{";", MakeBoxes[idx, StandardForm]}]]],
   RowBox[{"CD", "[", MakeBoxes[num, StandardForm], ",",
     MakeBoxes[den, StandardForm], "]"}]];

(* PARTIAL DERIVATIVES (The Comma Operator) *)

MakeBoxes[Partials[num_, den_], StandardForm] :=
  If[MatchQ[num, Partials[_, _]],

   (* CASE 1: Nested Derivative → Fraction *)
   Replace[num,
    Partials[top_, bot1_] :>
     FractionBox[
      RowBox[{SuperscriptBox["∂", "2"], MakeBoxes[top, StandardForm]}],
      RowBox[{RowBox[{"∂", MakeBoxes[den, StandardForm]}],
        RowBox[{"∂", MakeBoxes[bot1, StandardForm]}]}]]],

   (* CASE 2 & 3 *)
   If[MatchQ[den, x[𝒰[_]]] && ! MatchQ[num, x[𝒰[_]]],

    (* CASE 2: Comma Notation *)
    Replace[den,
     x[𝒰[idx_]] :>
      SubscriptBox[
       (* THE v1.6.1 FIX: Wrap in parentheses if Product/Sum/Power *)
```

```
        If[MemberQ[{Times, Plus, Power}, Head[num]],
         RowBox[{"(", MakeBoxes[num, StandardForm], ")"}],
         MakeBoxes[num, StandardForm]],

        (* Comma and Index *)
        RowBox[{",", MakeBoxes[idx, StandardForm]}]]],

    (* CASE 3: Default → Fraction *)
    FractionBox[
     RowBox[{"∂", MakeBoxes[num, StandardForm]}],
     RowBox[{"∂", MakeBoxes[den, StandardForm]}]] ] ];

MakeBoxes[Γ[𝒰[u_], 𝒟[d1_], 𝒟[d2_]], StandardForm] :=
  SubsuperscriptBox["Γ",
   RowBox[{MakeBoxes[d1, StandardForm], MakeBoxes[d2, StandardForm]}],
    MakeBoxes[u, StandardForm]];

MakeBoxes[δ[𝒰[up_], 𝒟[down_]], StandardForm] :=
  SubsuperscriptBox["δ", MakeBoxes[down, StandardForm],
   MakeBoxes[up, StandardForm]];

MakeBoxes[δ[𝒟[down_], 𝒰[up_]], StandardForm] :=
  SubsuperscriptBox["δ", MakeBoxes[down, StandardForm],
   MakeBoxes[up, StandardForm]];

MakeBoxes[h_[indices__], StandardForm] /; (h =!= δ) &&
(h =!= Partials) &&
(h =!= CD) &&
(h =!= Γ) &&
MatchQ[{indices}, {(_[𝒰] | _[𝒟] | 𝒰[_] | 𝒟[_]) ..}] :=
 Module[{formattedScripts},
  formattedScripts = {indices} /. {𝒰[i_] :>
      SuperscriptBox["",
       MakeBoxes[i, StandardForm]], 𝒟[i_] :>
      SubscriptBox["", MakeBoxes[i, StandardForm]]};
  RowBox[Prepend[formattedScripts, MakeBoxes[h, StandardForm]]]]

Protect[MakeBoxes];

(* ========================================================================*)
(* 4. ALGEBRAIC SIMPLIFICATION *)
(* ========================================================================*)

CanonicalizeTerm[term_] :=
  Module[{indices, counts, dummies, replacementRules},
   indices = Cases[term, (𝒰 | 𝒟)[i_] :> i, Infinity];
   counts = Tally[indices];
```

```
   dummies = Select[counts, Last[#] == 2 &][[All, 1]];
   replacementRules = MapIndexed[
     #1 → Symbol["i" <> ToString[First[#2]]] &,
     dummies];
   term /. replacementRules];

CanonicalizeIndices[expr_Plus] :=
  Total[CanonicalizeIndices /@ List @@ expr];

CanonicalizeIndices[expr_Equal] :=
  Equal @@ (CanonicalizeIndices /@ List @@ expr);

CanonicalizeIndices[expr_] := CanonicalizeTerm[expr];

robustTransformRules = {
  A_[𝒰[primed_]] :>
    Module[{fresh = Unique["μ"]},
     Partials[x[𝒰[primed]], x[𝒰[fresh]]]*A[𝒰[fresh]]],
  p_[𝒟[primed_]] :>
    Module[{fresh = Unique["ν"]},
     Partials[x[𝒰[fresh]], x[𝒰[primed]]]*p[𝒟[fresh]]]};

(* DYNAMIC TENSOR FORM *)
TensorForm[expr_] :=
  Module[{canonExpr, formalPattern,formalIndices,
          usedSymbols, greekPool, availableGreek, indexMap},

   (* 1. Canonicalize first *)
   canonExpr = CanonicalizeIndices[expr];

   (* 2. Define Formal Pattern: Any symbol starting with a...z *)
   (* This covers s, i1, i2, etc. *)
   formalPattern = CharacterRange["a", "z"];

   (* 3. Identify ALL Formal Indices *)
   formalIndices = Sort @ DeleteDuplicates @ Cases[canonExpr,
     s_Symbol /; StringStartsQ[SymbolName[s], formalPattern],
     Infinity];

   (* 4a. Identify symbols ALREADY in the expression *)
   usedSymbols = DeleteDuplicates @ Cases[canonExpr, _Symbol, Infinity];

   (* 4b. Define our preferred Greek pool *)
   greekPool = {λ, κ, ρ, σ, μ, ν, τ, η, χ, ψ, α, β, γ};

   (* 5. Filter the pool: Remove symbols that are already used *)
```

```
    availableGreek = DeleteElements[greekPool, usedSymbols];

    (* 6. Map the ordered, found formals to the available Greek letters *)
    indexMap = Thread[formalIndices → Take[availableGreek, UpTo[Length[formalIndices]]]];

    (* 7. Apply *)
    HoldForm[Evaluate[canonExpr /. indexMap]] ];

(* EXTRACT COEFFICIENT FIELD (Quotient-Theorem Tool) *)
ExtractCoefficient[expr_, field_Symbol] :=
 Module[{processTerm, expanded, termList},
  expanded = Expand[expr];
  termList = If[Head[expanded] === Plus,
    List @@ expanded,
    {expanded}];

  processTerm[term_] := Module[{canonTerm, dummyIndex, targetIdx, normalizedTerm},
    (* 1. Canonicalize locally *)
    canonTerm = CanonicalizeIndices[term];

    (* 2. Find the index attached to the field *)
    dummyIndex = Cases[canonTerm, field[_[idx_]] :→ idx, Infinity];
    If[dummyIndex === {}, Return[0]];

    (* 3. NORMALIZE: Swap the vector's index to a Reserved Symbol (FormalS) *)
    (* This prevents it from colliding with internal dummies (i1, i2) in other terms *)
    targetIdx = First[dummyIndex];
    normalizedTerm = canonTerm /. targetIdx → Symbol["s"];

    (* 4. Extract coefficient of A^S *)
    Coefficient[normalizedTerm, field[𝒰[Symbol["s"]]]] +
    Coefficient[normalizedTerm, field[𝒟[Symbol["s"]]]]];

  Total[processTerm /@ termList]];

(* ========================================================================*)
(* 5. APPLICATION-SPECIFIC RULES *)
(*    User must explicitly apply these when wanted *)
(* ========================================================================*)

metricRules = {
  g[𝒟[mu_], 𝒟[nu_]]*vec_[𝒰[nu_]] :→ vec[𝒟[mu]],
  vec_[𝒰[nu_]]*g[𝒟[mu_], 𝒟[nu_]] :→ vec[𝒟[mu]],

  g[𝒟[nu_], 𝒟[mu_]]*vec_[𝒰[nu_]] :→ vec[𝒟[mu]],
  vec_[𝒰[nu_]]*g[𝒟[nu_], 𝒟[mu_]] :→ vec[𝒟[mu]],
```

```
  g[𝒰[mu_], 𝒰[nu_]]*covec_[𝒟[nu_]] :> covec[𝒰[mu]],
  covec_[𝒟[nu_]]*g[𝒰[mu_], 𝒰[nu_]] :> covec[𝒰[mu]],

  g[𝒰[nu_], 𝒰[mu_]]*covec_[𝒟[nu_]] :> covec[𝒰[mu]],
  covec_[𝒟[nu_]]*g[𝒰[nu_], 𝒰[mu_]] :> covec[𝒰[mu]],

  g[𝒰[mu_], 𝒰[α_]]*g[𝒟[α_], 𝒟[nu_]] :> δ[𝒰[mu], 𝒟[nu]],
  g[𝒟[α_], 𝒟[nu_]]*g[𝒰[mu_], 𝒰[α_]] :> δ[𝒰[mu], 𝒟[nu]],

  (* δ-Contraction Rules (For Associativity) *)

  (* Rule: T^β * Subscript[δ^α, β] → T^α *)
  expr_ * δ[𝒰[b_], 𝒟[a_]] /; !FreeQ[expr, 𝒟[b]] :>
    (expr /. 𝒟[b] → 𝒟[a]),
  δ[𝒰[b_], 𝒟[a_]] * expr_ /; !FreeQ[expr, 𝒟[b]] :>
    (expr /. 𝒟[b] → 𝒟[a]),

  (* Rule: Subscript[T, α * ]Subscript[δ^α, β] → Subscript[T, β] *)
  expr_ * δ[𝒰[a_], 𝒟[b_]] /; !FreeQ[expr, 𝒰[b]] :>
    (expr /. 𝒰[b] → 𝒰[a]),
  δ[𝒰[a_], 𝒟[b_]] * expr_ /; !FreeQ[expr, 𝒰[b]] :>
    (expr /. 𝒰[b] → 𝒰[a]),

  (* Cleanup: Delta on itself *)
  δ[𝒰[a_], 𝒟[b_]] * δ[𝒰[b_], 𝒟[c_]] :> δ[𝒰[a], 𝒟[c]]

  };

(* Torsion-Free Geometry (General Relativity) *)
(* User must apply this manually: expr //. torsionRules *)
torsionRules = {
  RelativityConnection[u_, a_, b_] :>
   RelativityConnection[u, Sort[{a, b}][[1]], Sort[{a, b}][[2]]]
};

(* ========================================================================*)
(* 6. DIFFERENTIATION ENGINE *)
(* ========================================================================*)

differentiationRules = {
  Partials[a_*b_, var_] :> Partials[a, var]*b + a*Partials[b, var],
  Partials[a_ + b_, var_] :> Partials[a, var] + Partials[b, var],
  Partials[expr_, x[𝒰[idx_]]] /; StringContainsQ[ToString[idx], "'"] :>
    Module[{fresh = Unique["σ"]},
     Partials[x[𝒰[fresh]], x[𝒰[idx]]]*
      Partials[expr, x[𝒰[fresh]]]]};
```

```
ExpandDerivatives[expr_] := expr //. differentiationRules;

(* Schwarz's Theorem (Sort Partial Derivatives) *)
(* If variables are out of canonical order, swap them *)
Partials[Partials[f_, x[𝒰[a_]]], x[𝒰[b_]]] /;
  !OrderedQ[{a, b}] :=
  Partials[Partials[f, x[𝒰[b]]], x[𝒰[a]]];

(* ========================================================================== *)
(* 7. COVARIANT DERIVATIVE                                                    *)
(* ========================================================================== *)

(* CONFIGURATION: The Connection Symbol *)
(* Default is Γ, but user can change it to A, C, etc., for Electrodynamics, Yang-Mills, ...
RelativityConnection = Γ;

SetConnection[sym_Symbol] := (
   RelativityConnection = sym;
   Print["Relativity Engine: Connection set to ", sym]
);

(* Rule 0: Connection-Binding (The "Gamma Glue") *)
(* DYNAMIC: Checks if prod contains the CURRENT RelativityConnection symbol *)
(* This treats 'Gamma * A' as a single atom to enforce correct scope *)
CD[prod_Times /; !FreeQ[prod, RelativityConnection], var_] :=
  Module[{up, down, partialTerm, upCorrections, downCorrections},

   {up, down} = valence[prod];
   partialTerm = Partials[prod, var];

   (* Apply Corrections using the CURRENT Connection Symbol *)
   upCorrections = Sum[
     Module[{lam = Unique["λ"]},
      RelativityConnection[𝒰[idx], 𝒟[var⟦1,1⟧], 𝒟[lam]] * (prod /. idx → lam)
     ], {idx, up}];

   downCorrections = Sum[
     Module[{lam = Unique["λ"]},
      RelativityConnection[𝒰[lam], 𝒟[idx], 𝒟[var⟦1,1⟧]] * (prod /. idx → lam)
     ], {idx, down}];

   partialTerm + upCorrections - downCorrections ];

(* Rule 1: Linearity *)
CD[a_ + b_, var_] := CD[a, var] + CD[b, var];
```

```
(* Rule 2: Product Rule *)
CD[a_ * b_, var_] := CD[a, var] * b + a * CD[b, var];

(* Rule 3: The General Tensor Rule (Master Rule) *)
CD[expr_, x[𝒰[nu_]]] :=
 Module[{up, down, partialTerm, upCorrections, downCorrections},

  {up, down} = valence[expr];
  partialTerm = Partials[expr, x[𝒰[nu]]];

  (* DYNAMIC: Generate corrections using RelativityConnection *)
  upCorrections = Sum[
    Module[{lam = Unique["λ"]},
     RelativityConnection[𝒰[idx], 𝒟[nu], 𝒟[lam]] * (expr /. idx → lam)
    ], {idx, up}];

  downCorrections = Sum[
    Module[{lam = Unique["λ"]},
     RelativityConnection[𝒰[lam], 𝒟[idx], 𝒟[nu]] * (expr /. idx → lam)
    ], {idx, down}];

  partialTerm + upCorrections - downCorrections ];
```

Smoke Tests for v1.6.1, Fix Display Bug

In[•]:=
```
Partials[A[𝒰[μ]], x[𝒰[ν]]]
```

Out[•]=

$$A^{\mu}{}_{,\nu}$$

In[•]:=
```
Partials[A[𝒰[μ]] * Γ[𝒰[r], 𝒟[a], 𝒟[b]], x[𝒰[ν]]]
```

Out[•]=

$$\left(\left(A^{\mu}\right) \Gamma^{r}{}_{a\,b}\right)_{,\nu}$$

Regression Tests

Visual Showcase

Covariant Derivatives

Law of Gamma

Connections to Christoffel Symbols

```
In[ ]:= Print["=========================================================================="];
Print["Christoffel Symbols for Gravitation from Metric via the \
𝒰𝒟 Calculus"];
Print["=========================================================================="];

ClearAll[g, GammaSymmetryRule, MetricSymmetryRule, term1, term2,
  term3, combination, targetIndex];

Print["1. PHYSICAL CONSTRAINTS ----------------------------------------------------"];
Print["1a. Torsion-free geometry: \
\!\(\*SubscriptBox[SuperscriptBox[\(Γ\), \(α\)], \
\(βγ\)]\) == \!\(\*SubscriptBox[SuperscriptBox[\(\
Γ\), \(α\)], \(γβ\)]\), \
[MTW] Box 10.2, page 250 "];
GammaSymmetryRule = Echo[
   Γ[u_, 𝒟[b_], 𝒟[c_]] :>
    Γ[
     u, 𝒟[Sort[{b, c}][[1]]], 𝒟[
      Sort[{b, c}][[2]]]],
   "Γ Symmetry: "];

Print["1b. Metric Symmetry: \!\(\*SubscriptBox[\(g\), \(α\
β\)]\) == \!\(\*SubscriptBox[\(g\), \(βα\)]\), \
[MTW] Equation 8.24, page 210"];
Print[Style[
   "(output manipulated as expected by 𝒰\
𝒟 display rules)", Gray]];
MetricSymmetryRule = Echo[
   g[𝒟[a_], 𝒟[b_]] :>
    g[𝒟[Sort[{a, b}][[1]]], 𝒟[
      Sort[{a, b}][[2]]]],
   "g Symmetry"];

Print["1c. Metric Compatibility Postulates \!\(\*SubscriptBox[\(\
∇\), \([λ\)]\)\!\(\*SubscriptBox[\(g\), \
\(μν\(]\)\)]\) = 0 [MTW] Eqn 8.23 & Chapter 14."];

(*Term 1: \!\(
\*SubscriptBox[\(∇\), \(λ\)]
\*SubscriptBox[\(g\), \(μν\)]\)*)
term1 = CD[g[𝒟[μ], 𝒟[ν]],
     x[𝒰[λ]]] /. GammaSymmetryRule /.
   MetricSymmetryRule;
Echo[TensorForm[term1],
```

```
  "0 = \!\(\*SubscriptBox[\(∇\), \
\(λ\)]\)\!\(\*SubscriptBox[\(g\), \(μν\)]\) =";

(*Term 2: \!\(
\*SubscriptBox[\(∇\), \(μ\)]
\*SubscriptBox[\(g\), \(νλ\)]\) (Left-Cycle Indices)*)
term2 = CD[g[𝒟[ν], 𝒟[λ]],
     x[𝒰[μ]]] /. GammaSymmetryRule /.
   MetricSymmetryRule;
Echo[TensorForm[term2],
  "0 = \!\(\*SubscriptBox[\(∇\), \(μ\)]\)\!\(\*SubscriptBox[\
\(g\), \(μλ\)]\) =";

(*Term 3: \!\(
\*SubscriptBox[\(∇\), \(ν\)]
\*SubscriptBox[\(g\), \(λμ\)]\) (Left-Cycle Indices)*)
term3 = CD[g[𝒟[λ], 𝒟[μ]],
     x[𝒰[ν]]] /. GammaSymmetryRule /.
   MetricSymmetryRule;
Echo[TensorForm[term3],
  "0 = \!\(\*SubscriptBox[\(∇\), \(ν\)]\)\!\(\*SubscriptBox[\
\(g\), \(λμ\)]\) =";

================================================================================
Christoffel Symbols for Gravitation from Metric via the 𝒰𝒟 Calculus
================================================================================
1. PHYSICAL CONSTRAINTS ------------------------------------------------
1a. Torsion-free geometry: Γ^α_βγ == Γ^α_γβ, [MTW] Box 10.2, page 250
» Γ Symmetry: Γ[u_, 𝒟[b_], 𝒟[c_]] :↦ Γ[u, 𝒟[Sort[{b, c}][[1]]], 𝒟[Sort[{b, c}][[2]]]]
1b. Metric Symmetry: g_αβ == g_βα, [MTW] Equation 8.24, page 210
(output manipulated as expected by 𝒰𝒟 display rules)
» g Symmetry g a_ b_ :↦ g a b
1c. Metric Compatibility Postulates ∇_[λ g_μν] = 0 [MTW] Eqn 8.23 & Chapter 14.
» 0 = ∇_λ g_μν = g μ ν,λ - (g κ ν) Γ^κ_λμ - (g κ μ) Γ^κ_λν
» 0 = ∇_μ g_μλ = g λ ν,μ - (g κ ν) Γ^κ_λμ - (g λ κ) Γ^κ_μν
» 0 = ∇_ν g_λμ = g λ μ,ν - (g κ μ) Γ^κ_λν - (g λ κ) Γ^κ_μν
```

```
In[•]:= Print["2. CYCLIC PERMUTATION TRICK ----------------------------------------------"];
        Print["Write 'Koszul' sum of ∇g permutations that isolates a \
        Γ term."];
        Print["[MTW] Equation 8.24b, page 210, in coordinate basis \
        (\!\(\*SubscriptBox[\(c\), \(abc\)]\)===0)"];
        Print["The Linear Combination 0 = (\!\(\*SubscriptBox[\(∇\), \(\
        μ\)]\)\!\(\*SubscriptBox[\(g\), \(νλ\)]\) + \
        \!\(\*SubscriptBox[\(∇\), \(ν\)]\)\!\(\*SubscriptBox[\(g\), \
        \(λμ\)]\) - \!\(\*SubscriptBox[\(∇\), \(λ\)]\
        \)\!\(\*SubscriptBox[\(g\), \(μν\)]\)) ==="];

        Echo[combination = (term2 + term3 - term1), "0 ="];

        Echo[gammaPart =
           Select[List @@ combination, ! FreeQ[#, Γ] &] // Total,
          "Γ part ="];
        Echo[gammaPartCanonical = gammaPart // CanonicalizeIndices,
          "Γ part (canonical) ="];
        Echo[metricPartials =
           Select[List @@ combination, FreeQ[#, Γ] &] // Total,
          "g partials"];
```

2. CYCLIC PERMUTATION TRICK ----------

Write 'Koszul' sum of ∇g permutations that isolates a Γ term.

[MTW] Equation 8.24b, page 210, in coordinate basis (c_{abc}===0)

The Linear Combination 0 = ($\nabla_\mu g_{\nu\lambda} + \nabla_\nu g_{\lambda\mu} - \nabla_\lambda g_{\mu\nu}$) ===

» 0 = $g_{\lambda\ \mu,\nu} + g_{\lambda\ \nu,\mu} - g_{\mu\ \nu,\lambda} + (g_{\lambda 17\ \nu})\,\Gamma^{\lambda 17}_{\lambda\mu} + (g_{\lambda 18\ \mu})\,\Gamma^{\lambda 18}_{\lambda\nu} - (g_{\lambda\ \lambda 19})\,\Gamma^{\lambda 19}_{\mu\nu} - (g_{\lambda 20\ \nu})\,\Gamma^{\lambda 20}_{\lambda\mu} - (g_{\lambda 21\ \mu})\,\Gamma^{\lambda 21}_{\lambda\nu} - (g_{\lambda\ \lambda 22})\,\Gamma^{\lambda 22}_{\mu\nu}$

» Γ part = $(g_{\lambda 17\ \nu})\,\Gamma^{\lambda 17}_{\lambda\mu} + (g_{\lambda 18\ \mu})\,\Gamma^{\lambda 18}_{\lambda\nu} - (g_{\lambda\ \lambda 19})\,\Gamma^{\lambda 19}_{\mu\nu} - (g_{\lambda 20\ \nu})\,\Gamma^{\lambda 20}_{\lambda\mu} - (g_{\lambda 21\ \mu})\,\Gamma^{\lambda 21}_{\lambda\nu} - (g_{\lambda\ \lambda 22})\,\Gamma^{\lambda 22}_{\mu\nu}$

» Γ part (canonical) = $-2\left(g_{\lambda\ j1}\right)\Gamma^{j1}_{\mu\nu}$

» g partials $g_{\lambda\ \mu,\nu} + g_{\lambda\ \nu,\mu} - g_{\mu\ \nu,\lambda}$

```
In[•]:= Print["3. SOLVE FOR Γ ------------------------------------------------------------"];

        (*Equation: gammaPart + metricPartials = 0 => -gammaPart = \
        metricPartials*)
        Print["3a. Equation to solve:"];
        lhsRaw = -gammaPartCanonical;
        rhsRaw = metricPartials;

        Echo[TensorForm[lhsRaw == rhsRaw], "Isolated Equation:"];

        Print["3b. Find correct \!\(\*SuperscriptBox[\(g\), \(-1\)]\) to \
        multiply through."];
```

```
(*1. Find the metric inside the LHS term*)
Echo[metricFound = First[Cases[lhsRaw, g[__], Infinity]], "g found"];

(*2. Find the Gamma inside the LHS term*)
Echo[gammaFound = First[Cases[lhsRaw, Γ[__], Infinity]],
   "Γ found"];

(*3. Extract raw symbols from the indices*)
Echo[metricIndices = First /@ (List @@ metricFound), "g indices"];
Echo[gammaUpIndex = First[First[gammaFound]],
  "Γ up index"];

(*4. Identify the Target Index*)
(*The g index that matches Γ up-index is the Dummy. \
The other is the Target.*)
Echo[targetIndex = If[metricIndices[[1]] === gammaUpIndex,
    metricIndices[[2]], metricIndices[[1]]], "target index"];

(*5. Construct the specific Inverse needed*)
Echo[invMetricFactor =
   1/2*g[𝒰[σ], 𝒰[targetIndex]],
   "\!\(\*FractionBox[\(1\), \(2\)]\)\!\(\*SuperscriptBox[\(g\), \(-1\
\)]\) ="];
Print[Style[
   "(1/2 cancels factor 2 from Γ part (canonical) above)",
    Gray]];

Print["3c. Multiply and Contract"];

(*δ-Γ\
 contraction rule (will be promoted into the engine in next installmen\
t)*)
Echo[deltaContractionRule = δ[𝒰[
       s_], 𝒟[r_]]*Γ[𝒰[
       r_], a_, b_] :> Γ[𝒰[s], a, b],
  "δ-Γ contraction rule: "];
Print[Style[
   "(will be promoted into 𝒰𝒟 in the \
future)", Gray]];

(*Execute the algebra*)
Echo[lhsSolved =
   lhsRaw*invMetricFactor //. metricRules //. deltaContractionRule,
  "LHS solved: "];

(*Format RHS:Factor out 1/2 Subscript[g^σ, λ]*)
Echo[rhsSolved =
```

```
  Collect[rhsRaw*invMetricFactor,
   g[𝒰[_], 𝒰[_]]],
  "RHS solved:"];

(*Step D: Final Result*)
Print["\n=== CHRISTOFFEL SYMBOLS FOR GRAVITATION ==="];
Print["aka The Fundamental Theorem of Riemannian Geometry"];
finalEquation = lhsSolved == rhsSolved;
TensorForm[finalEquation]
```

3. SOLVE FOR Γ ----------

3a. Equation to solve:

» Isolated Equation: $2\,(g_{\lambda\,\kappa})\,\Gamma^{\kappa}_{\mu\nu} == g_{\lambda\,\mu,\nu} + g_{\lambda\,\nu,\mu} - g_{\mu\,\nu,\lambda}$

3b. Find correct g^{-1} to multiply through.

» g found $g_{\lambda\,j1}$

» Γ found $\Gamma^{j1}_{\mu\nu}$

» g indices $\{\lambda, j1\}$

» Γ up index $j1$

» target index λ

» $\frac{1}{2} g^{-1} = \frac{1}{2}\,(g^{\sigma\,\lambda})$

(1/2 cancels factor 2 from Γ part (canonical) above)

3c. Multiply and Contract

» δ–Γ contraction rule: `Γ[𝒰[r_], a_, b_] δ^s_r_ :> Γ[𝒰[s], a, b]`

(will be promoted into $\mathcal{UD}$ in the future)

» LHS solved: $\Gamma^{\sigma}_{\mu\nu}$

» RHS solved: $\frac{1}{2}\,(g^{\sigma\,\lambda})\,(g_{\lambda\,\mu,\nu} + g_{\lambda\,\nu,\mu} - g_{\mu\,\nu,\lambda})$

=== CHRISTOFFEL SYMBOLS FOR GRAVITATION ===

aka The Fundamental Theorem of Riemannian Geometry

Out[•]=

$$\Gamma^{\sigma}_{\mu\nu} == \frac{1}{2}\,(g^{\sigma\,\lambda})\,(g_{\lambda\,\mu,\nu} + g_{\lambda\,\nu,\mu} - g_{\mu\,\nu,\lambda})$$

The Commutator

In[*]:=
```
(* DEFINITION:The Commutator*)
(* ∇γ∇δA^α - ∇δ∇γA^α *)
Echo[term1 = CD[CD[A[𝒰[α]], x[𝒰[δ]]], x[𝒰[γ]]], "∇γ∇δA^β :"];
Echo[term2 = CD[CD[A[𝒰[α]], x[𝒰[γ]]], x[𝒰[δ]]], "∇δ∇γA^β :"];

Echo[commutator = term1 - term2,
  "raw commutator"];

(* STEP 1: Simplify *)
(* 1.1 Expand derivatives *)
 Echo[expanded = ExpandDerivatives[commutator],
  "xpd commutator :"];

Echo[tmp$ = expanded - commutator, "Expanded - SealedBox diagnostic :"];

(* 1.2 Expand algebraically and apply torsion rules *)
(* crucial to separate Γ*Partial from Γ*Γ *)
Echo[simplified = CanonicalizeIndices[expanded // Expand] //. torsionRules,
  "alg. xp + torsion elmin :"];

(* STEP 2: Verify Algebra *)
(* Does the second derivative of A vanish? *)
Echo[hasSecondDerivative = ! FreeQ[simplified, Partials[Partials[__]]],
  "test: has 2d derivs?"];

(* Apply Quotient Theorem *)
Echo[rawRiemann = ExtractCoefficient[simplified, A],
  "raw Riemann :"];

Print["\n=== THE RIEMANN CURVATURE TENSOR ==="];

(* 5. Readable Greek *)
Echo[TensorForm[rawRiemann], "R^α_λγδ ="];
```

» $\nabla_\gamma \nabla_\delta A^\beta$: $\frac{\partial^2 A^{\alpha}}{\partial x^{\delta} \partial x^{\gamma}} + \left(\left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\delta \lambda 23}\right)_{,\gamma} + \left(A^{\lambda 24}{}_{,\delta}\right) \Gamma^{\alpha}_{\gamma \lambda 24} -$
$\left(A^{\alpha}{}_{,\lambda 25}\right) \Gamma^{\lambda 25}_{\delta \gamma} + \left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\gamma \lambda 26} \Gamma^{\lambda 26}_{\delta \lambda 23} - \left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\lambda 27 \lambda 23} \Gamma^{\lambda 27}_{\delta \gamma}$

» $\nabla_\delta \nabla_\gamma A^\beta$: $\frac{\partial^2 A^{\alpha}}{\partial x^{\delta} \partial x^{\gamma}} + \left(\left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\gamma \lambda 28}\right)_{,\delta} + \left(A^{\lambda 29}{}_{,\gamma}\right) \Gamma^{\alpha}_{\delta \lambda 29} -$
$\left(A^{\alpha}{}_{,\lambda 30}\right) \Gamma^{\lambda 30}_{\gamma \delta} + \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\delta \lambda 31} \Gamma^{\lambda 31}_{\gamma \lambda 28} - \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\lambda 32 \lambda 28} \Gamma^{\lambda 32}_{\gamma \delta}$

» raw commutator $- \left(\left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\gamma \lambda 28}\right)_{,\delta} + \left(\left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\delta \lambda 23}\right)_{,\gamma} +$
$\left(A^{\lambda 24}{}_{,\delta}\right) \Gamma^{\alpha}_{\gamma \lambda 24} - \left(A^{\lambda 29}{}_{,\gamma}\right) \Gamma^{\alpha}_{\delta \lambda 29} - \left(A^{\alpha}{}_{,\lambda 25}\right) \Gamma^{\lambda 25}_{\delta \gamma} + \left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\gamma \lambda 26} \Gamma^{\lambda 26}_{\delta \lambda 23} -$
$\left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\lambda 27 \lambda 23} \Gamma^{\lambda 27}_{\delta \gamma} + \left(A^{\alpha}{}_{,\lambda 30}\right) \Gamma^{\lambda 30}_{\gamma \delta} - \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\delta \lambda 31} \Gamma^{\lambda 31}_{\gamma \lambda 28} + \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\lambda 32 \lambda 28} \Gamma^{\lambda 32}_{\gamma \delta}$

» xpd commutator : $- \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\gamma \lambda 28,\delta} + \left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\delta \lambda 23,\gamma} + \left(A^{\lambda 24}{}_{,\delta}\right) \Gamma^{\alpha}_{\gamma \lambda 24} -$
$\left(A^{\lambda 28}{}_{,\delta}\right) \Gamma^{\alpha}_{\gamma \lambda 28} + \left(A^{\lambda 23}{}_{,\gamma}\right) \Gamma^{\alpha}_{\delta \lambda 23} - \left(A^{\lambda 29}{}_{,\gamma}\right) \Gamma^{\alpha}_{\delta \lambda 29} - \left(A^{\alpha}{}_{,\lambda 25}\right) \Gamma^{\lambda 25}_{\delta \gamma} + \left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\gamma \lambda 26} \Gamma^{\lambda 26}_{\delta \lambda 23} -$
$\left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\lambda 27 \lambda 23} \Gamma^{\lambda 27}_{\delta \gamma} + \left(A^{\alpha}{}_{,\lambda 30}\right) \Gamma^{\lambda 30}_{\gamma \delta} - \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\delta \lambda 31} \Gamma^{\lambda 31}_{\gamma \lambda 28} + \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\lambda 32 \lambda 28} \Gamma^{\lambda 32}_{\gamma \delta}$

» Expanded – SealedBox diagnostic : $- \left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\gamma \lambda 28,\delta} + \left(\left(A^{\lambda 28}\right) \Gamma^{\alpha}_{\gamma \lambda 28}\right)_{,\delta} +$
$\left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\delta \lambda 23,\gamma} - \left(\left(A^{\lambda 23}\right) \Gamma^{\alpha}_{\delta \lambda 23}\right)_{,\gamma} - \left(A^{\lambda 28}{}_{,\delta}\right) \Gamma^{\alpha}_{\gamma \lambda 28} + \left(A^{\lambda 23}{}_{,\gamma}\right) \Gamma^{\alpha}_{\delta \lambda 23}$

» alg. xp + torsion elmin : $- \left(A^{i1}\right) \Gamma^{\alpha}_{i1 \gamma,\delta} + \left(A^{i1}\right) \Gamma^{\alpha}_{i1 \delta,\gamma} + \left(A^{i1}\right) \Gamma^{i2}_{i1 \delta} \Gamma^{\alpha}_{i2 \gamma} - \left(A^{i1}\right) \Gamma^{i2}_{i1 \gamma} \Gamma^{\alpha}_{i2 \delta}$

» test: has 2d derivs? False

» raw Riemann : $-\Gamma^{\alpha}_{s \gamma,\delta} + \Gamma^{\alpha}_{s \delta,\gamma} + \Gamma^{i2}_{s \delta} \Gamma^{\alpha}_{i2 \gamma} - \Gamma^{i2}_{s \gamma} \Gamma^{\alpha}_{i2 \delta}$

=== THE RIEMANN CURVATURE TENSOR ===

» $R^{\alpha}{}_{\lambda \gamma \delta}$ = $-\Gamma^{\alpha}_{\lambda \gamma,\delta} + \Gamma^{\alpha}_{\lambda \delta,\gamma} - \Gamma^{\alpha}_{\kappa \delta} \Gamma^{\kappa}_{\lambda \gamma} + \Gamma^{\alpha}_{\kappa \gamma} \Gamma^{\kappa}_{\lambda \delta}$